Secure Telemetry Link

With Authenticated Elliptic-Curve Diffie-Hellman

OPTION 6: DIFFIE-HELLMAN KEY EXCHANGE

Design and implement a secure Diffie-Hellman Key Exchange protocol that is resistant to man-in-the-middle attacks:

- Implement the Diffie-Hellman key exchange protocol.
- Allow two parties to establish a shared secret key securely over an insecure channel.
- Demonstrate the security properties of the Diffie-Hellman key exchange protocol, including:
  - Confidentiality: Ensure that the shared secret key remains confidential between Alice and Bob and is not accessible to any eavesdropper.
  - Integrity: Guarantee that the exchanged public keys and shared secret key remain unchanged and unaltered during transmission.
  - Authentication: Implement mechanisms to authenticate the identities of Alice and Bob to prevent impersonation or man-in-the-middle attacks.

Nicholas Battle (35619694)

ITO5163 Introduction to Cryptography for Cybersecurity

YouTube Video Link:

https://youtu.be/NflyI_Wr0Vc

1 Introduction

In aerospace and defence systems, the link between an on-board flight computer and a ground control station carries mission-critical telemetry, such as position, altitude and control parameters. If an attacker can read, modify or inject data on this channel, the impact can be severe. In practice, embedded and IoT systems have historically shipped with weak or no internodal links, leaving them attractive vulnerable to cyber-attacks (Hughes 2019). Protecting confidentiality, integrity and authentication of telemetry is therefore essential. In practice, such links often traverse shared networks or radio infrastructure and must be treated as insecure channels, requiring end-to-end protection.

This project implements a mutually authenticated Diffie-Hellman key exchange and a secure communication channel that simulates the encrypted link between a flight controller (Bob) and ground control (Alice). The design utilises Elliptic-Curve Diffie-Hellman (ECDH) as opposed to the classic finite-field Diffie-Hellman, because X25519 provides equivalent security with significantly smaller key sizes and faster operations, which is optimal for resource-constrained embedded avionics (Hughes 2019). On top of X25519, the system uses Ed25519 digital signatures for identity, HKDF-SHA256 for key derivation, AES-256-GCM for authenticated encryption and HMAC-SHA256 for key confirmation. Together, these mechanisms enable two endpoints to establish a shared secret over an insecure network and exchange telemetry while maintaining confidentiality, integrity and authentication.

2 Design Overview

2.1 System Architecture

The system is built around three programs and several shared modules. Alice (alice.py) represents the ground control station: it listens on a TCP port, runs the authenticated handshake as the server, derives session keys and then receives and logs decrypted telemetry frames. Bob (bob.py) represents the on-board flight computer: It connects to Ground Control, runs the handshake as the client and sends encrypted telemetry generated either by a simulator (for demonstration and testing purposes), or real ADS-C data from a parsed CSV file. Attacker (attacker.py) is a man in the middle proxy that sits between Alice and Bob, forwards JSON messages and deliberately tampers with keys, signatures, key-confirmation tags and ciphertext to demonstrate that attacks are detected and rejected by the system.

Responsibilities are separated into modules. The crypto_suite.py module encapsulates the cryptographic primitives (X25519 and Ed25519 operations, HKDF-SHA256, AES-256-GCM and HMAC-SHA256). Handshake.py defines HandshakeClient and HandshakeServer, which manage static Ed25519 identity keys, fresh X25519 ephemeral keys per session, a transcript of handshake messages and the derived session keys. Secure_channel.py implements the AES-256-GCM channel with per-message sequence numbers and Additional Authenticated Data (AAD) for replay and integrity protection. Telemetry_stream.py provides simulated or real ADS-C telemetry streams, as well as TelemetryLogger for logging decrypted frames. This modular design keeps cryptography, protocol logic and payload handling independent, which makes the code easier to read and extend. The separation of cryptographic operations from transport logic ensures the protocol can operate over any underlying channel, whether TCP for demonstration or RF/satellite links in production. The system architecture is illustrated in Appendix A.

2.2 Handshake Protocol

At the beginning of each session, Alice and Bob generate fresh X25519 key pairs and compute an ECDH shared secret. Each endpoint also maintains persistent Ed25519 signing key pairs, generated once by a setup script. Alice signs a message containing hers and Bob's ephemeral public keys and a server specific context string, whereas Bob signs an analogous message with his and Alice's ephemeral keys and a client specific context string. Each side then verifies the signature using the peer's static Ed25519 public key.

To mitigate an attacker substituting their own identity key, Alice and Bob perform key pinning. They load the peer's expected static public key from the keys directory and compare it byte for byte with the value received in the handshake. If the keys do not match, the handshake is aborted. Once the X25519 shared secret is computed, both Alice and Bob concatenate it with SHA-256 hash of the handshake transcript and pass this into HKDF-SHA256. The 96-byte HKDF output is split into three 32-byte keys: k_app_bob_to_alice for encrypting telemetry from Bob to Alice, k_app_alice_to_bob for the reverse, and k_confirm for HMAC-based key confirmation.

Key confirmation utilises HMAC-SHA256 under k_confirm. Bob computes HMAC(k_confirm, transcript hash || "client") and sends the tag to Alice; Alice verifies it and then sends HMAC(k_confirm, transcript_hash || "server") back to Bob. If either verification fails, the session is terminated, proving that both parties derived the same key material from the same transcript.

Upon handshake completion, each endpoint constructs a SecureChannel using its send and receive keys. The channel maintains send_sequence and recv_sequence counters. For each message, it derives a 96 bit nonce from the sequence number, sets AAD to the string seq|N and uses AES-256-GCM to encrypt and authenticate a serialisable telemetry frame. The telemetry is treated as an opaque payload carried over the secure channel.

3 Implementation Details

3.1 Technologies and Packages

The system is implemented in Python, using the cryptography package for X25519 and Ed25519 primitives, AESGCM for AES-256-GCM, HKDF and HMAC-SHA256. Standard library modules support networking, parsing and file handling (e.g., socket, json, argparse, struct, csv). Pandas is used offline to preprocess large raw ADS-C dataset into smaller CSV, but is not required at runtime. Using well-tested library code instead of manually written cryptographic primitives reduces implementation risk and keeps the focus on protocol design.

3.2 Implementation Results and Robustness

With expected usage, Alice starts first, loads her Ed25519 key pair and Bob's pinned public key, and listens on a configurable host and port. When a TCP connection is accepted, Alice runs HandshakeServer, derives session keys, constructs a SecureChannel and begins decrypting and logging telemetry. Bob then loads his own key pair and Alice's pinned public key, connects to Alice and runs HandshakeClient. Once the handshake completes, Bob initialises a simulated or real telemetry stream and sends encrypted packets to the ground station (Alice). At the end of execution, both print channel statistics containing messages sent and received.

Error handling is designed to fail safely. If key files are missing or malformed, Alice and Bob exit with clear error messages. If the TCP connection fails, the client reports the error rather than

retrying indefinitely. During data transfer, if decryption or sequence checks fail, Alice logs a warning, increments a failure counter and closes the connection after too many consecutive failures. When attacker.py is inserted and modifies ephemeral keys, signatures, HMAC tags or ciphertext, verification fails and the handshake or subsequent messages are rejected. This demonstrates robustness against active attacks. Taken together, these behaviours show that the implementation meets the functional and security requirements of the assignment specification.

4 Security Analysis

4.1 Confidentiality

Confidentiality is provided by ephemeral X25519 key exchange, HKDF-based key derivation and AES-256-GCM. Each session uses fresh X25519 key pairs, so every ECDH shared secret and symmetric key is unique. Long term Ed25519 keys are used only for signatures; compromising a static key later does not reveal past session keys because those depend on ephemeral private values that are never transmitted. This provides forward secrecy; compromising static keys later cannot decrypt previously recorded sessions. HKDF-SHA256 takes the shared secret and transcript hash as input and derives independent application and confirmation keys in a one-way manner. AES-256-GCM then encrypts each frame with a unique 96 bit nonce derived from the sender's sequence number. The same key-nonce pair is never reused, which is required for the security of GCM. Network traffic captured will only show hexadecimal-encoded ciphertext and authentication tags; no plaintext telemetry fields are visible to passive observers, confirming that confidentiality is achieved in practice.

4.2 Integrity and Replay Protection

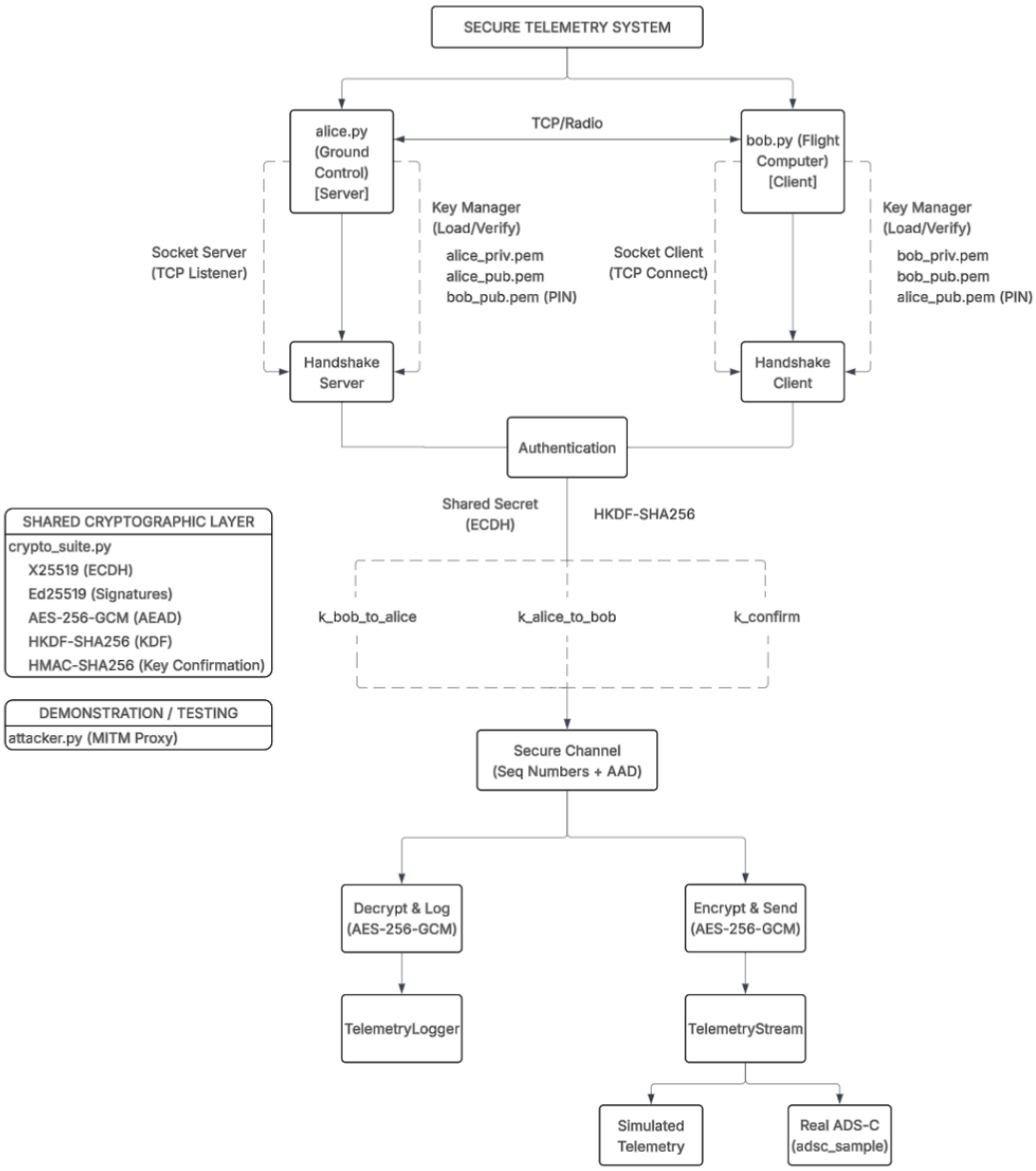
Integrity and replay protection are achieved through AES-GCMs authentication tag, combined with strict sequence number and AAD checks. For each outgoing message, SecureChannel encodes the current send_sequence as seq|n and uses this as AAD, which is covered by the GCM tag. On reception, the channel first checks that the received seq field equals the next expected recv_sequence, then reconstructs the expected AAD string and verifies the tag before returning plaintext. If any ciphertext, nonce or AAD is modified, GCM verification fails and no plaintext is returned. If an attacker replays an old ciphertext or injects a frame with the wrong sequence number, the "seq != recv_sequence" check fails and the message is rejected. This is where the sequence numbers directly restrict replay attacks; each ciphertext is bound to exactly one monotonic sequence value, and anything out of order or duplicated is discarded.

4.3 Authentication and Resistance to MITM Attacks

Authentication and Man-In-The-Middle resistance are provided by Ed25519 signatures, key pinning and HMAC-based key confirmation. Each party signs a message containing both ephemeral public keys and a role-specific context string, and verifies the peers signature using the pinned static public key. This binds the ephemeral ECDH keys to long-term, persistent identities and prevents an attacker from introducing their own. Key pinning ensures that the static public key observed at runtime matches a pre-provisioned value on disk; if an attacker presents a different identity key, the handshake fails immediately. HMAC-based key confirmation then proves that both sides derived the same key material from the same transcript before sending application data. The attacker.py proxy demonstrates that attempts to modify handshake messages, forge signatures or alter key confirmation tags cause verification to fail and the session to terminate. This shows that man in the middle attacks are detected and rejected.

Appendix

Appendix A: System Architecture



References

1. Hughes M (2019) *The Diffie-Hellman exchange in embedded cryptography - technical articles*, All About Circuits, accessed 20 November 2025, https://www.allaboutcircuits.com/technical-articles/the-diffie-hellman-exchange-in-embedded-cryptography/ .
2. Hughes M (2019) *How elliptic curve cryptography works - technical articles*, All About Circuits, accessed 29 November 2025, https://www.allaboutcircuits.com/technical-articles/elliptic-curve-cryptography-in-embedded-systems/
3. Olusoji O (2024) *Implementing diffie-hellman key exchange | by Obaloluwa Olusoji | Medium*, Medium, accessed 20 November 2025, https://medium.com/@olusojiobah/implementing-diffie-hellman-key-exchange-52c47367863a